

TECHNICAL REPORT

IoT / EMBEDDED SYSTEMS

Project 3: Motor Driver & Modular Code (OOP)

Daily Technical Report — Day 7

Platform ESP32-S3 · Arduino Framework · PlatformIO
Subsystem L298N Dual H-Bridge Motor Driver
Prepared For IoT / Embedded Systems Training Program

Confidential — Internal Training Documentatio

1. Daily Overview.....	3
1.1 Summary.....	3
1.2 Tasks Completed Today	3
2. Project 3: Motor Driver & Modular Code (OOP).....	3
2.1 Objective.....	3
2.2 Required File Structure	3
2.3 What to Build.....	4
1. config.h — Central Settings.....	4
2. Current Sensor Driver — C-Style Functions	4
3. L298N Motor Driver — C++ Class	4
4. main.cpp.....	5
2.4 PlatformIO Best Practices Checklist.....	5
3. L298N H-Bridge Wiring & Pinout Guide.....	5
3.1 Module Overview	5
3.2 Connection Matrix	5
1. Power & Ground Domain	5
2. Microcontroller Control Domain (ESP32-S3)	6
3. Load & Sensing Loop (Motor A).....	6
3.3 Pre-Power Verification Checklist	6
4. Schematics & Photographs	7
4.1 Circuit Schematic.....	7
4.2 Breadboard / Bench Photo — Overview.....	7
4.3 Close-up — L298N Wiring	Error! Bookmark not defined.
4.4 Close-up — Current Sensor & E-Stop Wiring.....	Error! Bookmark not defined.
4.5 Serial Monitor / Telemetry Capture	Error! Bookmark not defined.
5. Key Technical Concepts	8
5.1 Low-Pass Filter (LPF) Using an Exponential Moving Average (EMA).....	8
5.2 Interrupts in Embedded Systems	9
Why keep the ISR short	9
The volatile keyword	9
5.3 C++ enum vs. enum class	9
6. Open Questions / Socratic Checkpoints.....	10
7. Project Documentation & Notes	10
8. References.....	11

1. Daily Overview

1.1 Summary

This session advanced the motor-control platform from a single-file relay sketch into a modular, object-oriented PlatformIO project (Project 3). The DC motor driver was upgraded from the SRD-05VDC relay to an L298N dual H-bridge, enabling true speed (PWM) and direction control via ENA/IN1/IN2. Firmware was reorganized into header/source pairs — a C-style CurrentSensor module for ADC-based overcurrent detection, and a C++ class-based L298N driver for motor state and direction logic — governed by a central config.h.

On the signal-processing side, an Exponential Moving Average (EMA) low-pass filter was implemented to smooth noisy current-sensor readings without the memory cost of a full sample buffer. On the safety side, the emergency-stop button was moved from polling to a hardware interrupt (ISR) to guarantee immediate response, with the shared flag declared volatile to prevent compiler mis-optimization. Finally, enum class usage was adopted project-wide in place of traditional enum to gain scoped, strongly-typed state values for the motor and system state machines.

1.2 Tasks Completed Today

- Defined the Project 3 required file structure: config.h, CurrentSensor.h/.cpp, L298N.h/.cpp, main.cpp
- Built config.h as the single home for all pins, constants, and limits (L298N, current sensor, E-Stop, LEDs, current threshold, PWM frequency & resolution)
- Specified the CurrentSensor driver as plain C-style functions (Init, ReadAmps, IsOvercurrent) — no C++ class
- Designed the L298N C++ class (private pins/speed/direction; public constructor, begin, forward, reverse, stop, brake)
- Wired and documented the L298N H-bridge (power/ground, control, and load/sensing domains)
- Completed a pre-power verification checklist for the H-bridge and current-sensor wiring
- Implemented an EMA low-pass filter for current-sensor readings and validated it against raw ADC samples
- Converted the E-Stop input from polling to an ESP32 hardware interrupt (ISR) with a volatile flag
- Reviewed enum vs. enum class and standardized on enum class : uint8_t for system/motor states
- Planned main.cpp's safety guardrail: stop/brake + warning LED on E-Stop OR overcurrent

2. Project 3: Motor Driver & Modular Code (OOP)

2.1 Objective

Upgrade Project 2 by adding an L298N motor driver for speed/direction control, and refactor the existing code into clean, modular files.

2.2 Required File Structure

- config.h — central settings
- CurrentSensor.h & CurrentSensor.cpp — C-style module
- L298N.h & L298N.cpp — C++ class
- main.cpp — application entry point

```

my_project_folder/
├── platformio.ini      <- Hardware & environment configuration
├── include/           <- Declarations & interface header files
│   ├── config.h       <- Pin mappings, limits, PWM constants
│   ├── CurrentSensor.h <- C-style declarations
│   └── L298N.h        <- C++ class declaration
├── src/               <- Function & class implementations
│   ├── CurrentSensor.cpp <- C-style implementation
│   ├── L298N.cpp       <- Class method logic
│   └── main.cpp        <- Entry point (setup & loop)
├── lib/              <- Custom / third-party libraries (optional)
└── test/             <- Unit testing scripts (optional)

```

2.3 What to Build

1. config.h — Central Settings

Put ALL pin numbers, constants, and limits in this one file. Nothing hardcoded in .cpp files.

- Pins for the L298N, current sensor, E-Stop, and LEDs
- Current limit threshold (in amps)
- PWM frequency & resolution

2. Current Sensor Driver — C-Style Functions

Constraint

Do NOT use a C++ class for this module. Use standard, free-standing functions.

```

void  CurrentSensor_Init(int pin, float sensitivity);
float CurrentSensor_ReadAmps(void);
bool  CurrentSensor_IsOvercurrent(float threshold);

```

3. L298N Motor Driver — C++ Class

Create an L298N class with the following blueprint:

Private:

- Pin numbers
- Current speed / direction

Public:

- Constructor: L298N(in1, in2, ena)
- begin()
- forward(speed)
- reverse(speed)

- stop()
- brake()

4. main.cpp

- Include config.h and initialize everything
- Run the motor and monitor current
- If E-Stop is pressed OR current goes over the limit, stop/brake the motor immediately and trigger the warning LED

Safety Rule

Trigger an immediate stop/brake and light the warning LED if Current > Threshold OR E-Stop is pressed.

2.4 PlatformIO Best Practices Checklist

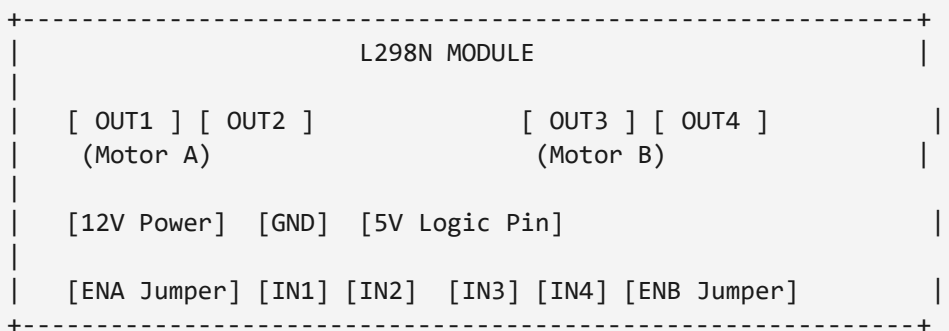
- Use #pragma once (or #ifndef / #define / #endif guards) at the top of every .h file
- Maintain clean separation: declarations live in .h, operational code lives in .cpp
- Include #include "config.h" in .cpp files whenever pin configurations or parameters are needed

3. L298N H-Bridge Wiring & Pinout Guide

Reference: lastminuteengineers.com — L298N DC / stepper driver Arduino tutorial (onboard 5V regulator and jumper section).

3.1 Module Overview

The L298N is a dual H-bridge motor driver designed to switch high-voltage / high-current DC motor loads using low-voltage logic signals from a microcontroller such as the ESP32-S3.



3.2 Connection Matrix

1. Power & Ground Domain

Critical Rule

Never omit the common ground connection. The ESP32-S3 and the external motor power supply must share a zero-volt ground reference.

L298N Pin	Connected To	Description / Purpose
12V Terminal	External motor power (+)	High-current power source for the DC motor (e.g., 6V–12V).
GND Terminal	External power (–) AND ESP32-S3 GND	Shared system ground reference.
5V Terminal	Unconnected (or 5V input if >12V supply)	Onboard 5V regulator output when the 5V jumper is engaged.

2. Microcontroller Control Domain (ESP32-S3)

L298N Pin	ESP32-S3 Pin Type	Function	Logic / Behavior
ENA	PWM-capable GPIO	Speed control	Controls motor duty cycle (0–100%) via PWM. Remove jumper cap first.
IN1	Digital output GPIO	Direction control 1	High/Low directional state line.
IN2	Digital output GPIO	Direction control 2	High/Low directional state line.

3. Load & Sensing Loop (Motor A)

Terminal	Path	Description
OUT1	→ ACS712 terminal 1	Current flows from OUT1 into the ACS712 current sensor.
ACS712 terminal 2	→ DC motor positive (+)	Sensor output passes through to the positive motor terminal.
OUT2	→ DC motor negative (–)	Completes the H-bridge return loop.

3.3 Pre-Power Verification Checklist

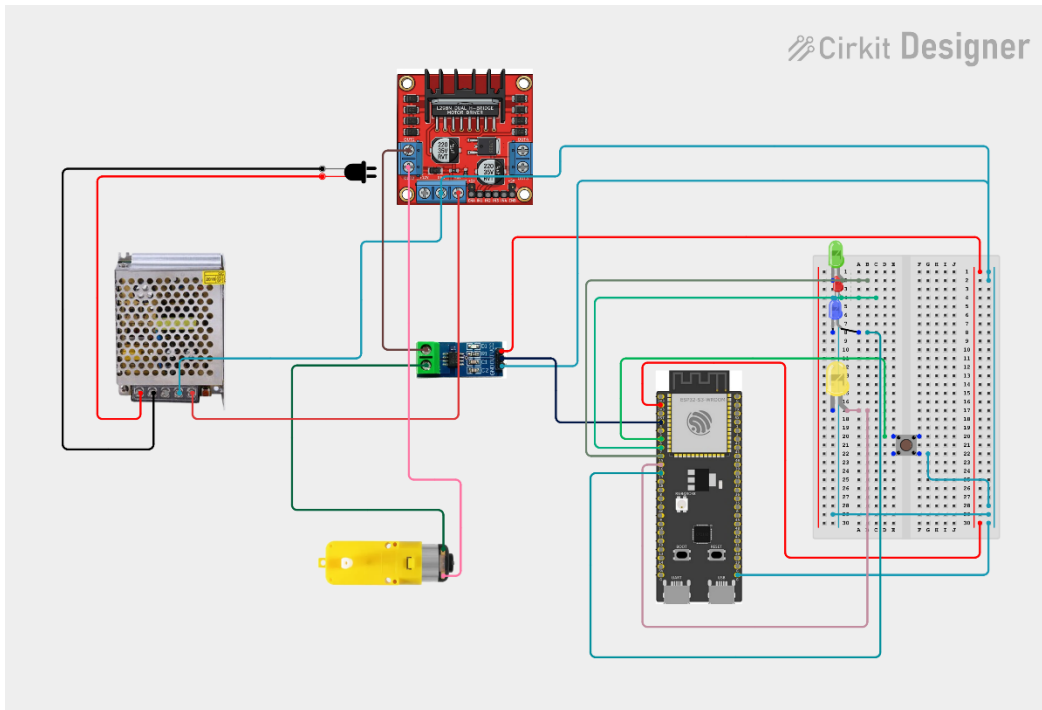
- Common ground — multimeter reads $\approx 0\ \Omega$ resistance between ESP32-S3 GND and L298N GND
- Series current path — the ACS712 sensor is wired in series on the OUT1 line, not across power and ground
- ENA jumper — physical jumper removed from ENA before attaching the ESP32-S3 PWM line
- Floating pins — direction pins IN1 and IN2 are explicitly mapped to digital output pins

4. Schematics & Photographs

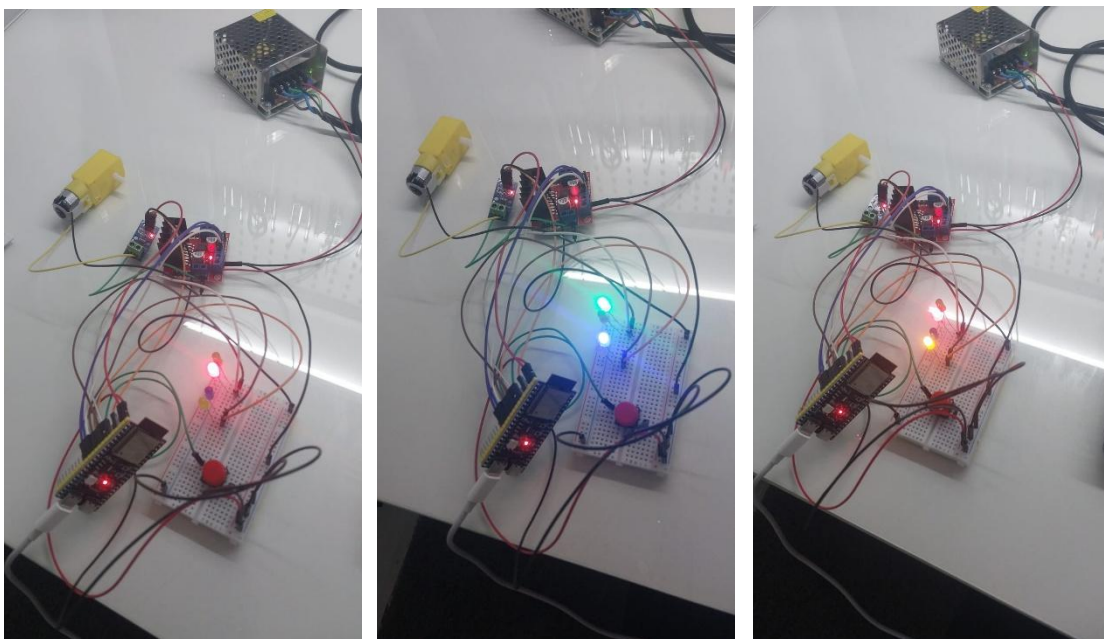
Insert Media Here

This section is a placeholder for the wiring schematic and bench photos for Project 3. Insert images below each heading (Insert > Pictures in Word), then delete this callout box.

4.1 Circuit Schematic



4.2 Photos



5. Key Technical Concepts

5.1 Low-Pass Filter (LPF) Using an Exponential Moving Average (EMA)

Sensor measurements from an ADC are affected by electrical noise, quantization, supply fluctuations, and EMI, causing fluctuating readings even when the true signal is constant. A low-pass filter attenuates this high-frequency noise while preserving genuine, slower changes. The Exponential Moving Average (EMA) is a first-order IIR filter that stores only one previous output value, making it ideal for embedded systems.

Standard equation:

$$y[n] = \alpha * x[n] + (1 - \alpha) * y[n-1]$$

Optimized (single-multiply) equivalent used in firmware:

```
filteredOutput +=
alpha * (newSample
- filteredOutput);
```

alpha (α)	Response Speed	Noise Reduction
0.05	Very slow	Excellent
0.10	Slow	High
0.20	Balanced	Good
0.50	Fast	Moderate
1.00	Immediate	No filtering

For the ESP32 current-sensing application, alpha = 0.1–0.3 provides a good balance between stability and responsiveness. The filter state variable must be declared static so the previous output persists between calls:

```
static float filteredCurrent = 0.0f;
const float alpha = 0.2f;

float CurrentSensor_ReadAmps()
{
    // ... average 50 ADC samples into 'voltage' ...
    float current = (voltage - CURRENT_SENSOR_OFFSET) / CURRENT_SENSOR_SCALE;
    filteredCurrent += alpha * (current - filteredCurrent);
    return filteredCurrent;
}
```

This combination provides two levels of noise reduction: sample averaging reduces random ADC noise within one measurement, and EMA filtering smooths successive measurements over time — producing a stable current reading suitable for real-time overcurrent detection.

5.2 Interrupts in Embedded Systems

An interrupt pauses normal program execution to immediately respond to an important event by running an Interrupt Service Routine (ISR); execution then resumes exactly where it left off. This avoids polling the E-Stop pin every loop iteration, lowering CPU usage and guaranteeing immediate response for safety-critical functions.

```
attachInterrupt(
    digitalPinToInterrupt(E_STOP_PIN),
    emergency_stop_isr,
    FALLING
);
```

Why keep the ISR short

Long ISRs delay other interrupts, reduce responsiveness, and complicate debugging. The recommended pattern is to have the ISR only set a flag, and let loop() perform the actual work:

```
volatile bool emergencyStop = false;

void IRAM_ATTR emergency_stop_isr()
{
    emergencyStop = true;    // minimal work inside the ISR
}

void loop()
{
    if (emergencyStop) {
        emergencyStop = false;
        motor.stop();
        Leds_Control(SystemState::EMERGENCY_STOP);
    }
}
```

The volatile keyword

volatile tells the compiler a variable can change unexpectedly outside normal program flow (e.g., inside an ISR). Without it, the compiler may cache the variable in a register and never observe the ISR's update, causing the main loop to hang indefinitely. Any flag shared between an ISR and the main program must be declared volatile.

- Keep the ISR as short as possible
- Avoid delay(), dynamic memory allocation, and Serial.print() inside an ISR
- Use volatile for variables shared between the ISR and the main program
- On the ESP32, mark ISRs with IRAM_ATTR so they execute reliably even when Flash is busy

5.3 C++ enum vs. enum class

Traditional enum places its values in the surrounding (global) scope, converts implicitly to int, and allows values from unrelated enums to be compared or assigned — a source of naming conflicts and silent bugs. enum class (C++11) scopes values inside the enum name, requires explicit casting to convert to an integer, and prevents cross-enum assignment errors.

Feature	enum	enum class
Scope	Global	Inside enum
Type safety	Weak	Strong
Integer conversion	Automatic	Requires casting
Name conflicts	Possible	Prevented
Embedded usage	Acceptable	Recommended

Best practice adopted for this project — a fixed-width, strongly typed state enum:

```
enum class SystemState : uint8_t
{
    STOP,
    RUNNING,
    FAULT_OVERCURRENT,
    EMERGENCY_STOP
};

SystemState state = SystemState::RUNNING;
```

Specifying `: uint8_t` reduces `sizeof(SystemState)` from the default 4 bytes down to 1 byte — meaningful savings on RAM-constrained microcontrollers such as the ESP32.

6. Open Questions / Socratic Checkpoints

Carried forward from the Project 3 kickoff session for follow-up before Step 1 & 2 are finalized:

1. Which exact GPIO pins on the ESP32-S3 will be assigned to IN1, IN2, and ENA, and do they support hardware PWM output?
2. If pin definitions are placed as `const int PIN_X = 5;` inside `config.h` (without `constexpr` or `#define`), what linker error occurs when multiple `.cpp` files include that header?
3. How should `config.h` be structured so pin numbers and calibration parameters cannot be accidentally redefined across multiple `.cpp` files?
4. If IN1 = HIGH and IN2 = HIGH while ENA receives a 100% PWM signal, what physically happens to current in the motor coils, and how does this differ from IN1 = LOW and IN2 = LOW?

7. Project Documentation & Notes

- `#define` vs `const` vs `constexpr` in Embedded C++ (reference notes)
- L298N.cpp — class method implementation notes
- ESP32 Motor Control System — full code breakdown
- Low-pass filters for embedded systems — background reading (investing.com EMA primer; smowcode and dmcinfo DSP/filter blog posts)

. References

Sources consulted during this session, in order of use:

- [In-Depth: Interface L298N DC Motor Driver Module with Arduino](#)
- [Beginner's Guide to Using Exponential Moving Averages \(EMA\) - Investing.com](#)
- [Exponential Moving Averages Explained Simply In 2 Minutes](#)
- [Applying Filters in Embedded Technology - Smowcode Blogs](#)
- [Simplifying DSP Filters for Embedded Systems | DMC, Inc.](#)
- [Day 5 + Day 7](#)

End of Report